Desafio Técnico - Desenvolvedor Fullstack Jr | Credifit

Prazo

Esperamos receber sua solução até às 23h59 da data especificada no e-mail. Se precisar de mais tempo, é só responder o e-mail sugerindo uma nova data. Caso não seja viável para nós, te avisamos e alinhamos uma alternativa juntos.

Sobre a Credifit

A Credifit é uma fintech que oferece crédito consignado para trabalhadores de empresas parceiras. O funcionário solicita um empréstimo e paga as parcelas via desconto direto na folha de pagamento, sem boleto, sem atraso.

Recentemente, o governo integrou o crédito consignado ao sistema e-Consignado do Dataprev, criando a modalidade chamada Crédito do Trabalhador. Com isso, antes de conceder um empréstimo, precisamos:

- Verificar se a empresa está habilitada no Dataprev
- O funcionário precisa autorizar a consulta dos seus dados
- Consultar o score de crédito e a margem disponível via Dataprev
- Processar o pagamento via um gateway externo

Seu desafio é construir a API que orquestra esse fluxo.

O que você vai construir

Uma API REST em Node.js/TypeScript que implementa o fluxo completo de solicitação de crédito do trabalhador. O fluxo é:

1. Cadastrar empresa conveniada

2. Cadastrar funcionário (vinculado à empresa)

3. Funcionário autoriza consulta ao Dataprev (válida por 24h)

4. Consultar margem consignável

5. Simular empréstimo (opções de 1 a 4 parcelas)

6. Solicitar empréstimo (verifica score, margem, limite e processa pagamento)

7. Listar e consultar empréstimos

Cada etapa corresponde a um endpoint da API. O contrato completo (URLs, payloads, status codes) está no arquivo api-contract.yaml que você recebe junto a este documento.

Dica: para visualizar o contrato de forma interativa, copie o conteúdo do api-contract.yaml e cole em https://editor.swagger.io, lá você consegue ver todos os endpoints, schemas e exemplos de forma visual.

Importante: sua API será validada por um script de testes automatizados que bate nos endpoints e verifica rotas, status codes, nomes dos campos e regras de negócio. Por isso, siga o contrato à risca. Rotas, nomes de campos nos payloads e status codes de resposta devem ser exatamente como definidos no api-contract.yaml.

O que você precisa entregar

Um único arquivo .zip contendo:

- Código-fonte da API em TypeScript + Node.js (framework à sua escolha: NestJS, Express, Fastify, etc.)
- Banco de dados PostgreSQL com a modelagem que você definir
- Testes automatizados cobrindo as regras de negócio
- Dockerfile da sua aplicação
- docker-compose.yml funcional

Ao rodar o comando abaixo, toda a stack deve subir e a API deve ficar acessível em http://localhost:3000:

```
docker compose up --build
```

Quando terminar: envie a solução em um único arquivo .zip para processo-seletivo@credifit.com.br, respondendo o e-mail do desafio. Não inclua node_modules nem artefatos de build no arquivo.

O que você recebe (starter kit)

Para que você possa focar na API, já entregamos a infraestrutura pronta:

Arquivo	O que é	Para que serve
api-contract.yaml	Especificação OpenAPI	Define todos os endpoints, request/response e status codes que sua API deve seguir
docker-compose.yml	Docker Compose	Sobe o PostgreSQL e os mocks já configurados, você só adiciona o serviço da sua API
mocks/	Servidor de mocks	Simula o Dataprev e o Gateway de Pagamento para você desenvolver sem depender de serviços reais

Como começar

```
# 1. Suba o banco e os mocks
docker compose up -d db mocks

# 2. Verifique que os mocks estão rodando
curl http://localhost:3001/health
# Deve retornar: {"status":"ok","scenario":"healthy"}

# 3. Desenvolva sua API na porta 3000

# 4. Quando pronto, descomente o bloco "api" no docker-compose.yml,
#    crie seu Dockerfile e teste:
docker compose up --build
```

Simulando falhas (cenários do mock)

O servidor de mocks pode simular falhas dos serviços externos — use isso para testar o tratamento de erros da sua API (o script de avaliação também usa esses cenários):

```
# Ver o cenário atual
curl http://localhost:3001/admin/scenario

# Alterar o cenário
curl -X POST http://localhost:3001/admin/scenario \
  -H "Content-Type: application/json" \
  -d '{"name": "gateway-down"}'
```

Cenário	Efeito
healthy	Tudo funcionando (cenário padrão)
dataprev-down	Dataprev responde HTTP 503
dataprev-slow	Dataprev demora 5 segundos para responder (testa o seu timeout)
gateway-down	Gateway responde HTTP 503
gateway-rejected	Gateway responde HTTP 200 com { "status": "recusado" }

Além dos cenários, o mock tem respostas determinísticas para facilitar seus testes:

- Empresa: CNPJs terminados em 0000 não estão habilitados no e-Consignado; todos os outros estão
- Score: CPFs terminados em 00 → score 300; terminados em 50 → score 500; terminados em 99 → score 900; demais → score 650

Variáveis de ambiente

Sua API deve ler a configuração destas variáveis de ambiente. Os valores mudam dependendo de onde você está rodando:

Durante o desenvolvimento (API rodando fora do Docker, direto na máquina):

Variável	Valor
DATABASE_URL	postgresql://credifit:credifit123@localhost:5432/credifit_credito
DATAPREV_BASE_URL	http://localhost:3001/dataprev
GATEWAY_BASE_URL	http://localhost:3001/gateway
PORT	3000

Na entrega final (tudo dentro do Docker Compose):

Variável	Valor
DATABASE_URL	postgresql://credifit:credifit123@db:5432/credifit_credito
DATAPREV_BASE_URL	http://mocks:3001/dataprev
GATEWAY_BASE_URL	http://mocks:3001/gateway
PORT	3000

Dentro do Docker Compose, os serviços se comunicam pelo nome do serviço (db, mocks), não por localhost. Os valores para Docker já estão pré-configurados no bloco comentado do docker-compose.yml.

Endpoints da API

Abaixo um resumo dos endpoints. O contrato completo está no api-contract.yaml, consulte-o para ver os payloads exatos, campos obrigatórios e status codes esperados.

Método	Rota	O que faz
POST	/empresas	Cadastra uma empresa conveniada
POST	/funcionarios	Cadastra um funcionário vinculado a uma empresa
POST	/funcionarios/:id/autorizacao-consulta	Funcionário autoriza consulta ao Dataprev
GET	/funcionarios/:id/margem	Retorna a margem consignável disponível
POST	/emprestimos/simular	Simula opções de parcelamento (1 a 4x)
POST	/emprestimos	Solicita um empréstimo (roda todas as validações)
GET	/emprestimos	Lista empréstimos (com filtros opcionais)
GET	/emprestimos/:id	Detalha um empréstimo com suas parcelas

credifit

Regras de negócio

1. Cadastros

- Empresa: CNPJ (14 dígitos), razão social, nome do representante, CPF do representante (11 dígitos), e-mail, senha.
- Funcionário: nome completo, CPF (11 dígitos), e-mail, senha, salário, ID da empresa.
- Unicidade: CPF, CNPJ e e-mail devem ser únicos em todo o sistema — inclusive entre entidades diferentes: o CPF do representante de uma empresa não pode ser igual ao CPF de um funcionário, e o e-mail de uma empresa não pode ser igual ao e-mail de um funcionário (e vice-versa). Se já existirem, retorne erro 409.
- Um funcionário só pode ser vinculado a uma empresa já cadastrada. Se a empresa não existir, retorne erro 404.

2. Autorização de consulta ao Dataprev

Antes de qualquer operação de empréstimo, o funcionário precisa autorizar a consulta dos seus dados.

- Endpoint: POST /funcionarios/:id/autorizacao-consulta
- A autorização expira em 24 horas
- Chamar o endpoint novamente renova a autorização: uma nova janela de 24 horas começa a contar a partir da nova chamada
- Se o funcionário tentar consultar margem, simular ou solicitar empréstimo sem autorização ativa, retorne erro 403

3. Verificação da empresa no Dataprev

Ao consultar margem (GET /funcionarios/:id/margem) e ao solicitar empréstimo (POST /emprestimos), verifique se a empresa do funcionário está habilitada no e-Consignado.

- Consultar o mock: GET {DATAPREV_BASE_URL}/empresa/{cnpj}/status
- Retorno: { "cnpj": "...", "habilitada": true/false }
- Se a empresa não estiver habilitada, retorne erro 422

- A simulação (POST /emprestimos/simular) não consulta o Dataprev — a margem é calculada localmente (regra 4)

4. Margem consignável

A margem define quanto o funcionário ainda pode comprometer do salário:

```
Margem total      = 35% do salário
Margem utilizada  = soma das parcelas mensais dos empréstimos ativos
(status "aprovado")
Margem disponível = margem total - margem utilizada
```

Exemplo: funcionário ganha R$ 5.000.

- Margem total = R$ 1.750 (35%)
- Tem 1 empréstimo ativo de R$ 500 em 2x = parcela de R$ 250/mês
- Margem utilizada = R$ 250
- Margem disponível = R$ 1.500

Regra: o valor solicitado de um novo empréstimo não pode exceder a margem disponível. (Como a parcela é sempre uma fração do valor solicitado, isso garante automaticamente que a parcela mensal também cabe na margem.)

Quando o valor excede a margem disponível:

- Na simulação (POST /emprestimos/simular) → retorne erro 400
- Na solicitação (POST /emprestimos) → o empréstimo é salvo com status 'rejeitado' (resposta 201), com motivoRejeicao explicando

5. Score de crédito
Ao solicitar um empréstimo, o sistema consulta o score do funcionário no Dataprev.

- Consultar o mock: GET {DATAPREV_BASE_URL}/funcionario/{cpf}/score
- Retorno: { "cpf": "...", "score": 650 }

Regra de aprovação - o score mínimo depende da faixa salarial:

Faixa salarial	Score mínimo
Até R$ 2.000,00	400
De R$ 2.000,01 até R$ 4.000,00	500
De R$ 4.000,01 até R$ 8.000,00	600
De R$ 8.000,01 até R$ 12.000,00	700
Acima de R$ 12.000,00	800

Se o score for insuficiente, o empréstimo é rejeitado.

6. Limite de empréstimos ativos
Cada funcionário pode ter no máximo 3 empréstimos com status 'aprovado' ao mesmo tempo. Se já tiver 3, novos empréstimos são rejeitados.

7. Parcelamento
- Opções: 1, 2, 3 ou 4 parcelas
- Valor da parcela = valor solicitado / número de parcelas (arredondar para 2 casas decimais)
- Todas as parcelas têm o mesmo valor (o resultado da divisão arredondado). Não ajuste a última parcela: para R$ 1.000,00 em 3x, as três parcelas valem R$ 333,33 — a diferença de centavos em relação ao valor solicitado é esperada e não precisa ser compensada
- Sem juros

8. Datas de vencimento das parcelas

- A parcela N vence N meses após a data de solicitação (1ª parcela: 1 mês depois; 2ª: 2 meses depois; e assim por diante)
- Se o mês de vencimento não tiver o dia correspondente, use o último dia do mês — mesma semântica do addMonths de bibliotecas como date-fns

Exemplo: empréstimo de 4 parcelas solicitado em 05/05/2025:

Parcela	Vencimento
1a	05/06/2025
2a	05/07/2025
3a	05/08/2025
4a	05/09/2025

Exemplo (dia de borda): empréstimo de 3 parcelas solicitado em 31/01/2025: vencimentos em 28/02/2025, 31/03/2025 e 30/04/2025.

9. Pagamento via Gateway

Quando todas as validações passam (score, margem, limite), o sistema processa o pagamento:

- Chamar o mock: POST {GATEWAY_BASE_URL}/pagamento
- Body: { "valor": 1000.00, "funcionarioCpf": "12345678900" }
- Se o gateway retorna { "status": "aprovado" } → empréstimo salvo com status 'aprovado'
- Se o gateway retorna { "status": "recusado" } (HTTP 200) → empréstimo salvo com status 'rejeitado' e motivoRejeicao "Pagamento recusado pelo gateway"
- Se o gateway falhar (HTTP 503 ou timeout) → empréstimo salvo com status 'erro_processamento'

Importante: em todos os casos acima a API responde 201 com o empréstimo salvo. Se o gateway estiver fora do ar, o empréstimo NÃO pode ser perdido — salve-o com o status 'erro_processamento' para que possa ser reprocessado depois. (O status HTTP 502 é reservado para indisponibilidade do Dataprev — veja a regra 11.)

10. Status do empréstimo

Todo empréstimo tem um destes status:

Status	Significado
aprovado	Todas as validações passaram E pagamento processado com sucesso
rejeitado	Alguma regra de negócio não foi atendida (score insuficiente, margem estourada, limite atingido) ou pagamento recusado pelo gateway
erro_processamento	Todas as regras passaram, mas o gateway de pagamento falhou (HTTP 503 ou timeout)

Empréstimos com status "rejeitado" devem incluir o campo "motivoRejeicao" explicando a razão (ex: "Score insuficiente para a faixa salarial").

11. Falhas de serviços externos

- Aplique um timeout de 3 segundos nas chamadas ao Dataprev e ao Gateway (o cenário dataprev-slow do mock demora 5 segundos — seu timeout deve disparar)
- Dataprev indisponível (HTTP 503 ou timeout): a operação não pode continuar — retorne erro 502 com mensagem clara. Nenhum empréstimo é salvo nesse caso
- Gateway indisponível (HTTP 503 ou timeout): o empréstimo já passou por todas as validações — salve-o com status 'erro_processamento' e responda 201 (regra 9)

O que será avaliado

Critério	O que olhamos
Funcionalidade	A API funciona conforme o contrato? Os endpoints retornam os payloads e status corretos?
Regras de negócio	Score, margem, limite, autorização. As validações estão corretas?
Modelagem do banco	As tabelas e relações fazem sentido para o domínio?
Tratamento de erros	O que acontece quando o Dataprev está fora? E o gateway? A API retorna erros claros?
Testes	Existem testes automatizados? Cobrem as regras principais?
Organização do código	O código está limpo, organizado em camadas, e fácil de entender?

O que NÃO será avaliado

- Frontend - não faça, o desafio é 100% backend
- Autenticação - não precisa implementar JWT, OAuth ou qualquer auth
- Deploy em nuvem - roda local com Docker e está ótimo
- Performance - não se preocupe com cache, filas ou otimização prematura

Checklist antes de enviar

- docker compose up --build sobe toda a stack sem erros
- API responde em http://localhost:3000
- Endpoints seguem o contrato (api-contract.yaml)
- Testes passam com npm test (ou equivalente)
- Solução compactada em um único arquivo .zip (sem node_modules nem artefatos de build), enviada para processo-seletivo@credifit.com.br

Boa sorte! Em caso de dúvidas, responda o e-mail do desafio.